

Apuntes Semana 13 Clase 1

Fabrizio Mena Mejía
Instituto Tecnológico de Costa Rica
IC-6200 Inteligencia Artificial
fabriciomena11@estudiantec.cr

Abstract—En este presente documento se van a presentar los apuntes de la clase del 19 de Mayo, correspondiente a la semana 13 del curso de Inteligencia Artificial, con el profesor Steven Pacheco. Se va a comenzar con comentarios y aspectos importantes con respecto al Proyecto1, luego como es habitual con un pequeño repaso de la clase anterior (Jueves 7 de Mayo) sobre el tema de la autoencoders y seguidamente se continua propiamente ya con los apuntes del día correspondiente, Martes 19 de Mayo, sobre la arquitectura de U-Net: Convolutional Network for Biomedical Image Segmentation.

I. COMENTARIOS SOBRE EL PROYECTO 1

La clase inicia con comentarios sobre las revisiones del proyecto 1, especifica que la rúbrica es muy importante revisarla para que los estudiantes salgan medianamente bien a la hora de ser evaluados.

Se brindan aspectos a mejorar, principalmente técnicos, probablemente son problemas de utilizar ias a la hora de realizar el proceso de escritura; es consciente de que los programadores las usan mucho, pero es deber de nosotros el poder revisar, leer y entender el contenido (hay que doble checkear). Hay que realizar una investigación a profundidad sobre lo que estamos redactando, y comprenderlo para poder defenderlo o debatirlo con personas que poseen ese conocimiento o explicárselo a las que no. Podemos aprovechar los outputs de la ia para evacuar dudas. En cuanto al reporte casi todos lo presentaron bien, los problemas más que todo son de estructura y orden, como se realizan los experimentos, los resultados como se presentan. No se deben de poner tantas páginas, con 8 páginas en las conferencias están bien, ya que a partir de esta cantidad se empieza a cobrar extra, así que para el proyecto realizar la sección del estado del arte es muy importante por lo que ahora es obligatoria para el siguiente proyecto, esta sección debe de ser parte del proceso de investigación, hacer las referencias en los textos es necesario, deben haber citas de párrafos o artículos que existan en medio. Por ejemplo hay que explicar la página de W&B, las gráficas generadas, explicar las funciones de las herramientas, la utilidad, que significan las gráficas, importantísimos los análisis de overfitting y underfitting.

II. EVALUACIONES

- Examen semana 18.
- Proyecto 2 se asigna semana 15.
- Se subieron las notas actualizadas.
- Quedan 3 tareas pendientes, pero por cuestiones de tiempo se dividen en 1 tarea normal, y una tarea grande pero que vale por 2.

III. REPASO

Este repaso sintetiza los conceptos más relevantes vistos en la clase anterior relacionados con autoencoders.

A. Autoencoders

Vamos a tener mecanismos de reconstrucción de datos (para la próxima tarea que tengamos vamos a tener reconstrucción de imágenes). Es no supervisado(Unsupervised learning) su objetivo es reconstruir la entrada original. La estructura del modelo que vamos a usar para reconstrucción de imágenes es la siguiente:

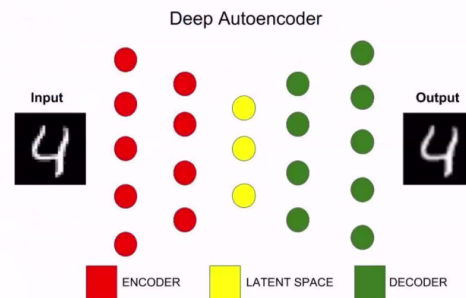


Fig. 1. Arquitectura de Autoencoder.

- 1) Encoder: Comprime la entrada en un vector latente. Extrae las características más importantes.
- 2) Espacio latente (cuello de botella): Representación comprimida de los datos. Contiene la información esencial
- 3) Decoder: Reconstruye la entrada a partir del vector latente.

Un autoencoder representa la entrada en un espacio, la idea es que el dato reconstruido sea lo mas similar a la entrada. Además hay que tener en cuenta que a pesar de ser no supervisado, no es cierto que no tenemos etiquetas. Formato de red, convierte la entrada a un vector latente, toma las características mas importantes de la imagen para pasarla a un espacio vectorial, luego se realiza un proceso inverso, y a partir del vector muy característico se intenta reconstruir la entrada.

B. Encoder

- Pueden verse como un conjunto de convoluciones seguidos de modulos de pooling.

- Se encarga de extraer las características principales de la imagen de entrada y de comprimir su información.
- Con esto se espera aprender la información más importante de la entrada.
- Se realiza Downsampling.

Es como lo que hemos estado haciendo de CNN pero ahora con un decoder en lugar de un clasificador.

C. Latent Space (Cuello de Botella)

Permite representar datos en un espacio reducido.

Facilita:

- Comparaciones semánticas entre datos.
- Reducción de dimensionalidad.

Ejemplo: Puntos cercanos en el espacio = datos similares.

El espacio latente es muy importante, así que hay que ser cuidadoso con la alimentación de datos.

Lo mas importante es la parte de cuello de botella, si no están tan bien distribuidos estos datos vamos a tener malas reconstrucciones.

D. Decoder

Es el proceso inverso a las convoluciones.

E. Reducción de dimensionalidad

Cuando hay una imagen de alta resolución y se codifica a un hiperplano, se toman esas características y las gráfica en un espacio. Es importante porque permite hacer el guardado en una versión compacta y hacer comparaciones semánticas para poder saber si dos imágenes se parecen. Si los puntos naranjas y rojos están muy cerca significa que pertenecen a una misma clase. Si se realizara este proceso basado en píxeles meramente sería demasiado complicado.

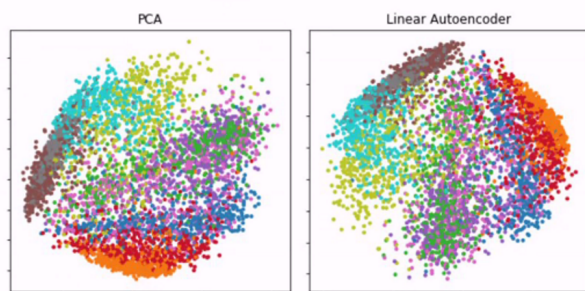


Fig. 2. PCA vs Autoencoder.

IV. DETECCIÓN DE ANOMALÍAS

La idea es que se realiza un entrenamiento para la reconstrucción de datos de una tarea tomando en cuenta su estado positivo.

El ejemplo que se había brindado es un sistema de detección de los voucher de sinpe móvil, para detectar si es verdadero o falso, se alimenta con 10 000 muestras un modelo y luego se pasa a un autoencoder para poder reconstruir de forma correcta las imágenes. Como solo se aprendió al ser entrenado con la clase correcta, si se le pasa un voucher incorrecto

va a generarse un vector latente erróneo y el proceso de reconstrucción va a ser de baja calidad. A partir de ahí se define un umbral para poder detectar cuando un voucher pertenece a la clase fraudulenta, también si son resultados bien altos de puede deducir a partir de eso si es correcto.

A. Procesamiento de imágenes

El objetivo es tener un autoencoder que permita remover ruido de las demás imágenes (salt and pepper por ejemplo), si lo alimentamos con imágenes con ruido, vamos a terminar entrenando un modelo capaz de eliminar ese ruido específico, pueden quitar blurs, pasar de blanco y negro a color, etc. Podemos entrar a la parte de super resolution (podría aparecer en una tarea futura). Esto abre muchas posibilidades.

B. Hiper parámetros a considerar

Tamaño de la codificación (vector latente).

- Decide cuanto se comprimen los datos.

Número de capas

- Profundidad del encoder y del decoder.
- Mayor = Modelo complejo.
- Menor = Modelo rápido.

Mientras más grande el vector latente más información de la imagen original, pero mientras más pequeño más comprimida la información de la imagen.

A mayor profundidad más hiper parámetros, y por ende más tiempo de entrenamiento.

Nosotros tenemos que encontrar el balance entre calidad y rendimiento.

Función de reconstrucción

- Altamente dependiente de la entrada y salida esperada.
- **MSE**: Cuantifica la "distancia" numérica promedio entre los píxeles (o valores) originales y los reconstruidos. Penaliza mucho más los errores grandes que los pequeños, obligando al modelo a ser muy preciso. Indica la calidad de reconstrucción de imágenes.
- **BinaryCrossEntropy**: Se utiliza cuando se trata la salida del autoencoder como si fuera una distribución de probabilidad.

Ventajas del BinaryCrossEntropy:

- **Tratamiento de píxeles**: Si los datos (como imágenes en escala de grises) tienen valores que representan una intensidad, se puede interpretar cada píxel como una probabilidad de "presencia" de ese punto.
- **Gradientes**: El BCE suele tener gradientes más "fuertes" cuando el modelo está muy lejos de la respuesta correcta. Esto puede ayudar a que el modelo aprenda a reconstruir bordes o detalles finos más rápido que con el MSE en ciertos tipos de datos.
- **Convergencia**: Muchos modelos (especialmente los Variational Autoencoders) prefieren BCE porque asumen que los píxeles siguen una distribución de Bernoulli (ceros y unos), lo cual a veces estabiliza el entrenamiento si la activación de la última capa es

una Sigmoide (que también comprime todo al rango $[0, 1]$).

C. Undercomplete Autoencoder

Es el que se encarga de condensar todo en un vector. Toma una imagen de entrada e intenta predecir la misma imagen de salida. La dimensión del espacio latente es menor que el espacio de entrada.

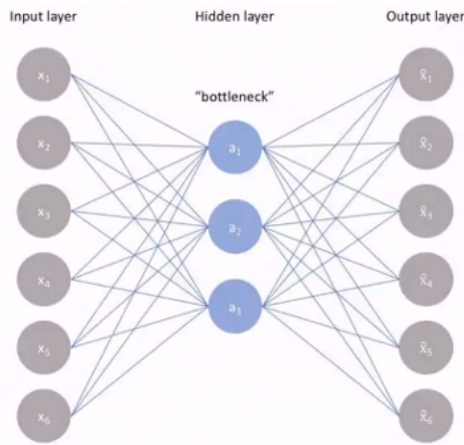


Fig. 3. Undercomplete Autoencoder.

D. Sparse Autoencoder

Este es el modelo diseñado específicamente para que, ante una entrada, solo un conjunto de neuronas esté activo (cercano a 1) y otra parte esté apagado (cercano a 0). Esto obliga a la red a encontrar las características más "esenciales" de los datos, ya que no puede usar todas las neuronas para memorizar cada detalle. Al igual que el cerebro, que no usa todas las neuronas al mismo tiempo, esto fuerza al modelo a aprender representaciones altamente interpretables y eficientes.

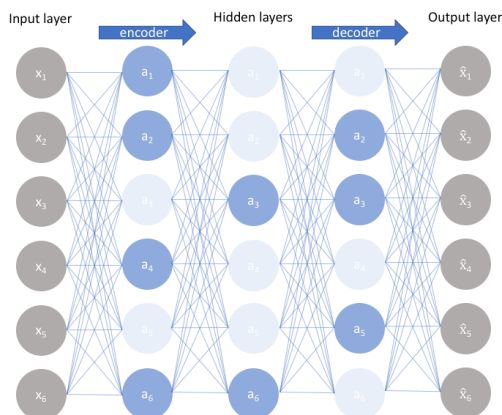


Fig. 4. Sparse Autoencoder.

E. Variational Autoencoder

Los autencoder estándar aprenden a representar un espacio latente, el problema es que este no es un espacio continuo, por lo que es difícil de **interpolar (estimar valores desconocidos dentro de un conjunto de datos conocidos)**.

- **Problema:** Valores del vector latente no están expresados de forma probabilística.
- **Solución:** Variational Autoencoders.
- **Resultado:** Expresa los valores como una distribución de probabilidad.

Si consigo hacer que las características se distribuyan en el espacio latente, puedo cambiar el género de una imagen pero que siga manteniendo su significado.

Por ejemplo, en la Figura 5 se pudo interpolar el espacio que

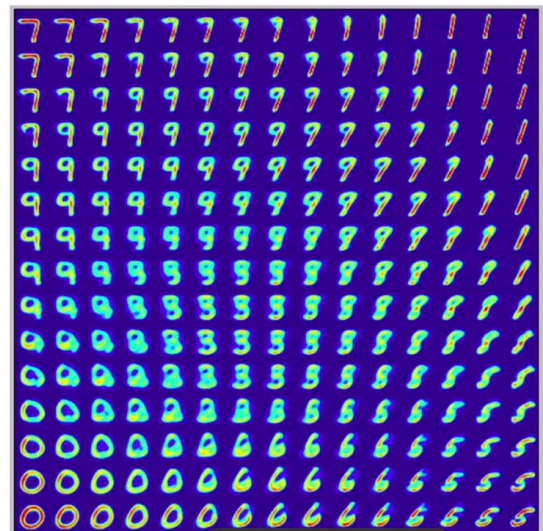


Fig. 5. Espacios entre clases.

hay entre 2 distintas clases(se pudo ver que hay entre ellas).

Lo que hacemos con el encoder es que tratamos de obtener la media y la desviación estándar para cada una de las clases que le ingresan al modelo, para que a partir de esa entrada se puede hacer una parametrización para poder obtener un espacio continuo.

Para eso usamos 2 funciones una para comparar si la imagen reconstruida se parece a la original y otra para comparar las distribuciones obtenidas.

Donde los resultados que difieran mas de la normal van a ser penalizados.

Proceso:

- Codificador produce 2 vectores para hacer la distribución:
 - La representación de μ .
 - La representación de σ estándar.
- Reparametrización:
 - Se toman muestras de μ y σ para producir el vector latente.

- Se toma al vector latente y se alimenta el decoder.
- Loss:
 - Disminuir la suma de BCE y Kullback-Leibler divergence (KLD).

A partir de BinaryCrossEntropy se optimiza la función de loss y la calidad de reconstrucción de la imagen.

V. MATERIA NUEVA: U-NET

Se propuso para hacer segmentación de imágenes en el dominio biomédico, debía de hacer máscaras de imágenes para detectar células; lo más importante es la arquitectura propuesta.

Tengo encoder y decoder como la arquitectura más vainilla:

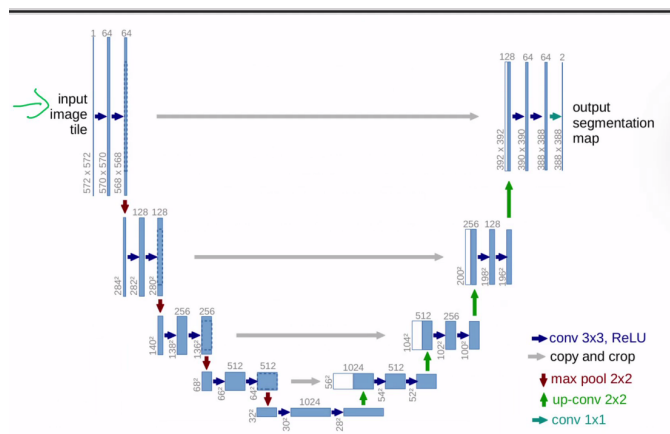


Fig. 6. Arquitectura de U-Net.

El upsampling que se aplica es de 512 pero dice 1024, pero ese proceso que se usa es el de hacer un skip-connection, hacemos el mismo proceso de autoencoding, pero por cada capa se tiene un skip-connection a cada capa que se tenía del encoder, esto permite preservar la información original. En u-net se producen las distintas máscaras para poder analizar la información que se desea de la imagen. Lo interesante de esta arquitectura es que hace uso de skip-connections para preservar mayor fidelidad.

La diferencia entre skip-connections y U-Net es que en el primero se hace una sumatoria y en el segundo se hace una concatenación.

A. Skip Connections

Sugiere que la salida de una capa anterior alimente la salida de una capa como la entrada de las siguientes capas. Resuelve el problema de degradación.

VI. TAREA 3

Se anuncia tarea 3 en Grupos. Vamos a empezar con los primeros pasos con ia generativa, vamos a comenzar con autoencoders para generar imágenes, no es tan pesada así que se puede usar collab ya que es con fines académicos, son arquitecturas pequeñas, pero vamos a comenzar a usar herramientas del framework de desarrollo de modelos de ia.

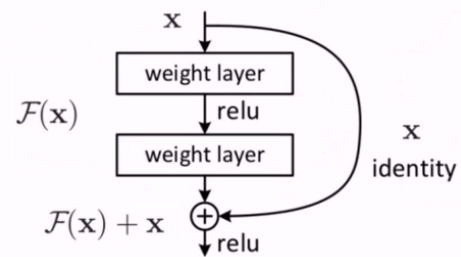


Fig. 7. Skip Connections.

- 1) La idea es darle más uso a pytorch lightning, que es como un pytorch con esteroides. Tiene muchísima personalización, se puede agregar fallbacks, bitácoras, etc. Hay que darse tiempo para explorar la herramienta. Hydra se puede dibujar como un sistema de anotaciones, para poder mapear de forma más ordenada las configuraciones de entrenamiento, con esta herramienta ya no están hardcoded, ahora se pueden escribir en .yaml y luego se cargan. Vamos a seguir con el uso de W&B, pero hay que registrar más cosas ahora.
- 2) El dataset que vamos a usar tiene un conjunto sano y otro de fallos, y lo que tratamos es a partir de reconocimiento de imágenes es un proceso de detección de anomalías, las que tienen defectos tienen funciones de pérdida más altas y un vector latente más alejado de lo esperado. Vamos a usar solo 4 clases esta vez para reducir el tiempo de entrenamiento. Para configuraciones el estudiante debe de usar Hydra. Se puede aumentar la estructura de los entrenamientos, se espera que no sea igual a la configuración de ejemplo si no que se investigue mas. Los módulos deberían de tener la información específica de cada entrenamiento, nada de esto debe de ir en el código. Vamos a hacer 8 entrenamientos usando L1 y L2, y aun no hemos visto SSIM y SSIM+L1. W&B vamos a usarlo como si fuera la BD de verdad. En el set de validación vamos a tener imágenes sanas, así que lo que vamos a ver es que nuestro modelo genera imágenes que tienen mayor sentido y van evolucionando con el tiempo, a partir de los ploteos podemos ir viendo como va mejorando o empeorando el vector latente, podemos agregar colores para representar mejor.
- 3) Se espera que para imágenes buenas se tenga una buena reconstrucción y que para las imágenes defectuosas se obtenga un mayor error y una representación latente diferente.
- 4) Todo el entregable presente en el notebook, no se usa Latex para que sea mas ligera la tarea. Histogramas de reconstrucción de imágenes. Debe de ir el paso a paso en el notebook, todas las imágenes deben de ser producidas por W&B, si se usa matplotlib, pone 0 el profe.

Nota: Hay algunas clases que son defectuosas pero parece

que se están confundiendo con las buenas, hay clases muy cercanas. Lo importante es que nosotros podamos ser el discriminador al observar este comportamiento y poder optimizarlo para que se produzca el mejor modelo.

A. Preguntas

No hace falta ponerse a investigar trabajos previos, lo importante es aprender a usar las herramientas.

Nosotros decidimos si cambiamos los grupos.

Aviso: Hay 2 lecturas en el tecDigital que las vamos a ocupar para la próxima semana Finding Universal Grammatical Relations in Multilingual BERT y On the Dangers of Stochastic Parrots. En las próximas clases vamos a hacer una pausa con la reconstrucción de imágenes y vamos a pasar con lo de lenguaje natural que es lo que se esta usando hoy en día

La primera trata sobre cómo y qué aprende mBERT (Multilingual BERT) sobre la estructura gramatical de los diferentes idiomas sin haber sido enseñado explícitamente para ello.

La segunda es una crítica a los modelos de lenguaje, son unas loras que repiten y repiten.

Próximo martes quiz puede venir información de los papers.